

Arduino 3S Li-ion BMS – Complete Engineering Project Guide

Project: 3-Cell Lithium-Ion Battery Management System

Platform: Arduino Uno (ATmega328P)

Author: Abhishek Varma | GITAM ECE | Batch 2025-29

Version: 1.0 | June 2026

Table of Contents

- [1. Project Overview](#)
 - [2. System Requirements](#)
 - [3. Hardware Design](#)
 - [4. Firmware Architecture](#)
 - [5. Implementation Plan](#)
 - [6. Testing and Validation](#)
 - [7. Engineering Documentation](#)
 - [8. GitHub Portfolio Strategy](#)
 - [9. Interview Preparation](#)
 - [10. Future Expansion](#)
-

1. Project Overview

1.1 Purpose

This project implements a Battery Management System (BMS) for a 3-cell series (3S) lithium-ion pack using an Arduino Uno. The system monitors individual cell voltages, pack temperature, detects fault conditions, drives status indicators, displays real-time data on an LCD, and logs measurements over UART.

The project is deliberately constrained to fundamental techniques – no BMS ICs, no dedicated monitoring chips – to build first-principles understanding of every measurement and decision the firmware makes. This is the same understanding required to work with production BMS ICs (TI BQ76920,

TI BQ76952, Maxim DS2782) because you must understand what the IC is doing at the register level to configure and debug it.

1.2 Learning Objectives

Objective	Concept
Design resistor divider networks for multi-cell voltage measurement	Analog signal conditioning
Implement ADC oversampling for improved measurement accuracy	Signal processing in firmware
Reconstruct individual cell voltages from cumulative tap measurements	Differential measurement
Calculate temperature from NTC thermistor using Steinhart-Hart B-parameter equation	Sensor physics and firmware math
Design a non-blocking state machine for alarm logic	Real-time embedded firmware patterns
Drive an I2C peripheral from scratch	I2C protocol implementation
Implement structured UART CSV logging	Communication protocol design
Apply engineering documentation standards	Professional practice

1.3 Engineering Concepts Demonstrated

Analog:

- Resistor voltage divider design with source impedance constraints
- ADC reference voltage and resolution
- 16x oversampling to improve effective resolution from 10-bit to 12-bit equivalent
- NTC thermistor behaviour (negative temperature coefficient, exponential resistance curve)
- Voltage reconstruction by subtraction from differential tap measurements

Digital/Firmware:

- Non-blocking timing using `millis()` (critical embedded pattern – never use `delay()` in production)
- I2C master driver implementation (PCF8574 GPIO expander driving HD44780 LCD)
- State machine for alarm management
- Circular buffer concept for averaging
- UART framing and CSV data structure

System:

- Fault detection thresholds with hysteresis to prevent alarm chatter
- Cell imbalance detection
- Graceful degradation under fault conditions
- Hardware/firmware co-design: what the hardware constrains, what firmware compensates

1.4 Relevance to BMS and Embedded Engineering

Every concept in this project maps directly to production BMS engineering:

This Project	Production BMS Equivalent
Resistor divider voltage measurement	BMS IC analog front-end (cell voltage inputs)
16x ADC oversampling	Delta-sigma ADC in BMS ICs (same principle, better hardware)
Differential tap subtraction	Cell stack measurement architecture
NTC thermistor	NTC array for pack thermal monitoring
UART CSV logging	CAN bus telemetry to vehicle BMS controller
LED alarm indicators	Contactor control, fault relay
Undervoltage threshold	Cell protection cutoff (hardware AND software)
Non-blocking state machine	FreeRTOS task in production BMS firmware

At Ather Energy, the BMS team writes firmware in C for STM32 microcontrollers using TI BQ76952 ICs. The questions they ask in interviews test exactly the concepts this project exercises: "How do you measure cell 2 voltage in a series stack?", "What happens to your measurement if the resistor tolerance is 5%?", "How do you prevent alarm chatter?"

2. System Requirements

2.1 Functional Requirements

ID	Requirement	Acceptance Criterion
FR-01	Measure individual cell voltages for all 3 cells	Measurement error < ± 50 mV across full operating range
FR-02	Measure pack temperature	Measurement error < $\pm 2^{\circ}\text{C}$ across 0-60 $^{\circ}\text{C}$ range

ID	Requirement	Acceptance Criterion
FR-03	Detect cell undervoltage condition	Alarm triggered within 100ms when any cell < 3.0V
FR-04	Detect cell overvoltage condition	Alarm triggered within 100ms when any cell > 4.25V
FR-05	Detect overtemperature condition	Alarm triggered within 500ms when temperature > 50°C
FR-06	Display cell voltages on LCD	Refresh rate $\geq$ 2 Hz, correct to 2 decimal places
FR-07	Indicate fault status via LEDs	Yellow LED per cell (green=normal, blink=warn, off=fault); Red LED for alarm
FR-08	Log measurements over UART	CSV format at 1 Hz, parseable by Python tool
FR-09	Calculate and display cell imbalance	Max-min cell voltage delta displayed when > 50 mV

2.2 Safety Requirements

ID	Requirement	Rationale
SR-01	Undervoltage alarm at 3.0V per cell	Li-ion cells below 3.0V suffer irreversible capacity loss. Below 2.5V: copper dissolution, internal short circuit risk.
SR-02	Overvoltage alarm at 4.25V per cell	Above 4.2V (typical), electrolyte oxidation accelerates. Above 4.35V: thermal runaway risk in some chemistries.
SR-03	Overtemperature alarm at 50°C	Li-ion degradation accelerates significantly above 45°C. At 60°C, SEI layer breakdown begins.
SR-04	Alarm state must persist until manually cleared or condition resolves	Prevents undetected fault re-entry
SR-05	ADC measurement must timeout gracefully	Firmware must not hang on ADC read failure
SR-06	All voltages above Arduino ADC range (>5V) must be divided before measurement	Protects ATmega328P ADC input pins (absolute max: VCC + 0.5V = 5.5V)

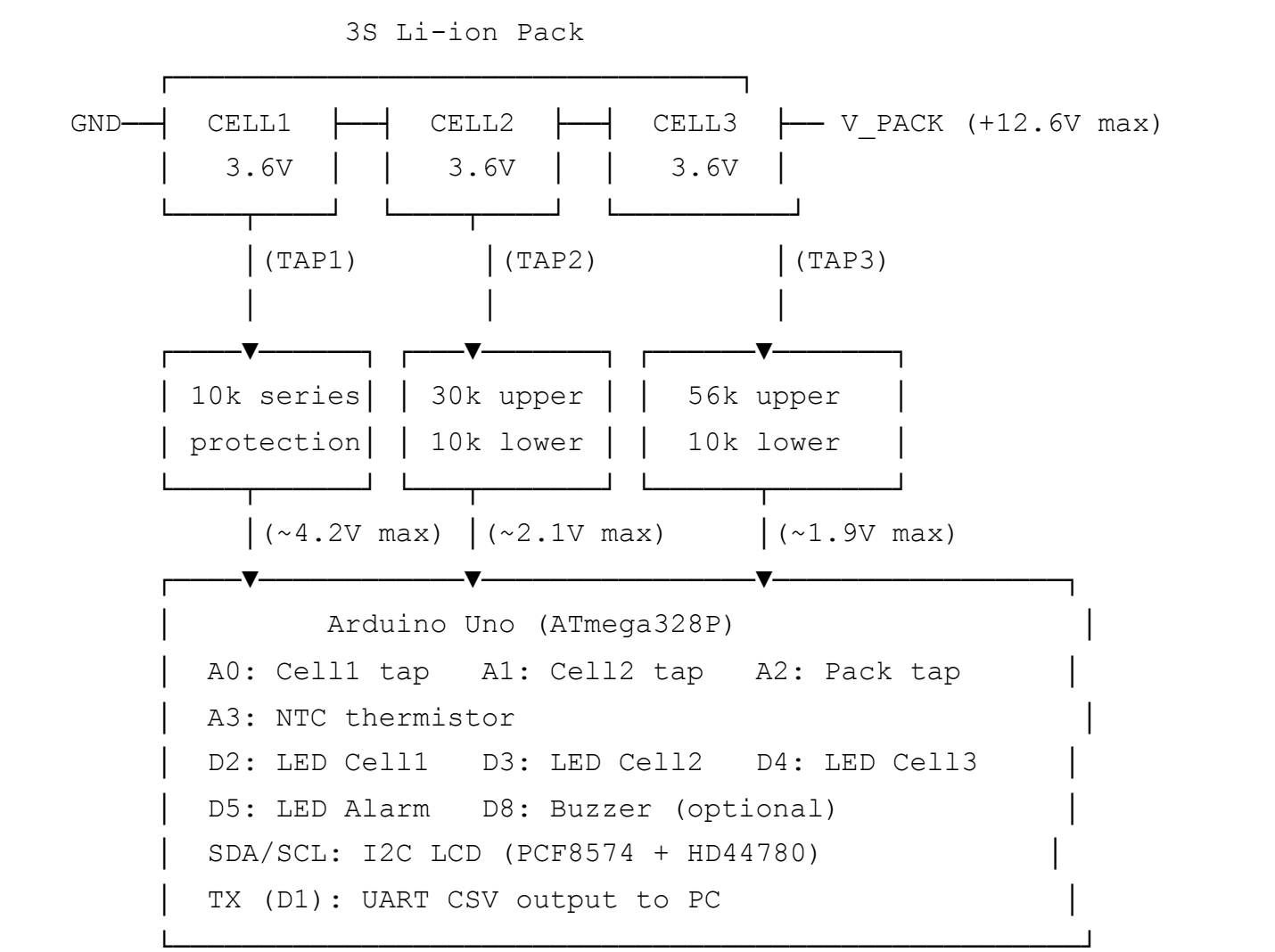
2.3 Performance Requirements

Parameter	Target	Rationale
-----------	--------	-----------

Parameter	Target	Rationale
Voltage measurement accuracy	$\pm 50\text{ mV}$	Sufficient to detect $\pm 100\text{ mV}$ cell imbalance
Temperature accuracy	$\pm 2^{\circ}\text{C}$	Sufficient for thermal alarm function
Measurement update rate	10 Hz raw, 1 Hz displayed/logged	Display stability; UART bandwidth
Alarm response time	< 200 ms from threshold crossing to indicator	Fast enough for protective function
UART log rate	1 sample/second	Manageable data rate; sufficient for trend analysis
ADC oversampling	16 samples per measurement	Improves effective resolution by ~ 2 bits

3. Hardware Design

3.1 Block Diagram



3.2 Complete Component List (BOM)

#	Component	Specification	Qty	Unit Cost (Rs)	Total (Rs)	Notes
1	Arduino Uno R3	ATmega328P, 16MHz, 5V	1	350	350	Or clone
2	18650 Li-ion cell	2000–3400 mAh, 3.7V nominal	3	180	540	Use matched cells from same batch
3	18650 cell holder (3S)	PCB mount or wire lead type	1	60	60	Or individual holders
4	16×2 I2C LCD	HD44780 controller + PCF8574 I2C backpack	1	120	120	Backpack address: 0x27 or 0x3F
5	NTC thermistor	10 k Ω at 25°C, B = 3950K	1	15	15	Epoxy-coated, waterproof preferred
6	Resistor 10 k Ω	1/4W, $\pm 1\%$ tolerance	5	1	5	Lower leg dividers + NTC pullup
7	Resistor 30 k Ω	1/4W, $\pm 1\%$ tolerance	1	1	1	Upper leg of TAP2 divider
8	Resistor 56 k Ω	1/4W, $\pm 1\%$ tolerance	1	1	1	Upper leg of TAP3 divider
9	LED Yellow 5mm	2.0V Vf, 20mA	3	5	15	One per cell indicator
10	LED Red 5mm	2.0V Vf, 20mA	1	5	5	Alarm indicator
11	Resistor 220 Ω	1/4W for LED current limiting	4	1	4	$I_{LED} = (5-2)/220 = 13.6$ mA
12	Capacitor 100 nF	Ceramic decoupling	4	3	12	One per ADC input pin
13	Breadboard	830 points	1	80	80	Or stripboard for permanent build
14	Jumper wires	Male-male, 20 cm	1 set	50	50	
15	USB cable	Type-A to Type-B for Arduino	1	60	60	UART connection to PC
				TOTAL	~1318 Rs	

Note: Use $\pm 1\%$ tolerance resistors for the voltage dividers. $\pm 5\%$ resistors introduce up to 10% error in the divider ratio, causing ~42 mV error at 4.2V — right at the margin of the overvoltage

3.3 Component Explanation

Arduino Uno (ATmega328P)

- 10-bit SAR ADC with 6 input channels on the Uno
- ADC reference: 5V (default), giving resolution = $5.0/1023 = 4.888$ mV/count
- ADC input impedance requirement: source impedance $< 10\text{ k}\Omega$ for accurate conversion (per datasheet Section 28.6)
- Digital I/O: 5V logic, 40 mA max per pin, 200 mA total
- I2C: SDA = A4, SCL = A5 (hardware I2C peripheral)
- UART: TX = D1, RX = D0 at 115200 baud

18650 Li-ion Cells

- Nominal voltage: 3.6–3.7V | Full charge: 4.2V | Cutoff: 2.5–3.0V
- 3S series pack: 10.8V nominal, 12.6V max, 9.0V cutoff
- Important: use cells from the same manufacturer, same batch, same age. Mismatched cells cause premature imbalance.

PCF8574 + HD44780 LCD

- PCF8574: 8-bit I2C GPIO expander. Converts I2C commands to 4-bit parallel HD44780 interface.
- Default I2C address: 0x27 (A0=A1=A2 = HIGH on PCF8574) or 0x3F (A0=A1=A2 = LOW on PCF8574A). Check your specific module.
- Backlight control: PCF8574 P3 pin drives NPN transistor for LCD backlight.
- Communication: 7-bit I2C address, 100 kHz standard mode (400 kHz Fast mode supported)

NTC Thermistor (10k Ω B3950)

- NTC = Negative Temperature Coefficient: resistance decreases as temperature increases
- At 25°C: $R = 10,000\ \Omega$. At 0°C: $R \approx 32,650\ \Omega$. At 80°C: $R \approx 1,669\ \Omega$
- Connected as lower leg of voltage divider ($R_{\text{pullup}} = 10\text{k}\Omega$ to 5V)
- Placement: thermally bonded to battery cell surface or holder

3.4 Voltage Divider Design – Full Calculations

Objective: Scale all tap voltages to the Arduino ADC range (0–5V).

Cell stack voltage ranges (referenced to GND):

- TAP1 (Cell 1 top): 0 to 4.2V $\rightarrow$ within ADC range, minimal divider needed
- TAP2 (Cell 2 top): 0 to 8.4V $\rightarrow$ must be divided

- TAP3 (Pack+, Cell 3 top): 0 to 12.6V → must be divided
-

TAP1 Circuit (A0 – measuring Cell 1 voltage):

Since max voltage = 4.2V < 5V (ADC limit), no divider is strictly required. However:

- A 10 k Ω series resistor is placed between TAP1 and A0
- This limits fault current if the battery is accidentally shorted to a high voltage rail
- The Arduino's internal ADC sample-and-hold capacitor (14 pF typ.) charged through this 10 k Ω requires ~1 μ s settle time – well within the ADC conversion time

Measurement equation:

$$V_{\text{cell1}} = \text{ADC0} \times (V_{\text{REF}} / 1023)$$

$$V_{\text{cell1}} = \text{ADC0} \times (5.0 / 1023)$$

At 4.2V: $\text{ADC0} = 4.2 \times 1023 / 5.0 = 860$ counts

At 3.0V: $\text{ADC0} = 3.0 \times 1023 / 5.0 = 614$ counts

Range used: 614–860 counts (246 counts for the cell operating range)

TAP2 Circuit (A1 – measuring cumulative voltage of Cell1 + Cell2):

Upper resistor: 30 k Ω (R_{upper})

Lower resistor: 10 k Ω (R_{lower})

Divider ratio: $k_2 = R_{\text{lower}} / (R_{\text{upper}} + R_{\text{lower}}) = 10,000 / 40,000 = 0.25$

Maximum voltage at A1: $8.4\text{V} \times 0.25 = 2.10\text{V}$ ✓ (< 5V)

Minimum voltage at A1: 0V

Maximum ADC1 count: $2.10 \times 1023 / 5.0 = 430$ counts

Reconstruction equation:

$$V_{\text{tap2_actual}} = \text{ADC1} \times (5.0 / 1023) \times (R_{\text{upper}} + R_{\text{lower}}) / R_{\text{lower}}$$

$$V_{\text{tap2_actual}} = \text{ADC1} \times (5.0 / 1023) \times (40,000 / 10,000)$$

$$V_{\text{tap2_actual}} = \text{ADC1} \times (5.0 / 1023) \times 4.0$$

Source impedance check (critical for ADC accuracy):

The ADC sees the parallel combination of R_{upper} and R_{lower}:

$$Z_{\text{source}} = 30\text{k} \parallel 10\text{k} = (30,000 \times 10,000) / (30,000 + 10,000) = 7,500 \Omega$$

Requirement: $Z_{\text{source}} < 10,000 \Omega$ ✓

(This is why the resistor values must not be made much larger – $300\text{k} \parallel 100\text{k} = 75 \text{ k}\Omega$ would violate the ADC requirement)

TAP3 Circuit (A2 – measuring full pack voltage):

Upper resistor: $56\text{ k}\Omega$ (R_{upper})

Lower resistor: $10\text{ k}\Omega$ (R_{lower})

Divider ratio: $k_3 = 10,000 / 66,000 = 0.1515$

Maximum voltage at A2: $12.6\text{V} \times 0.1515 = 1.910\text{V}$ ✓ ($< 5\text{V}$)

Maximum ADC2 count: $1.910 \times 1023 / 5.0 = 391$ counts

Reconstruction equation:

$$V_{\text{tap3_actual}} = \text{ADC2} \times (5.0 / 1023) \times (56,000 + 10,000) / 10,000$$

$$V_{\text{tap3_actual}} = \text{ADC2} \times (5.0 / 1023) \times 6.6$$

Source impedance check:

$$Z_{\text{source}} = 56\text{k} \parallel 10\text{k} = (56,000 \times 10,000) / 66,000 = 8,485\ \Omega < 10,000\ \Omega \checkmark$$

Cell voltage reconstruction:

$$V_{\text{cell1}} = V_{\text{tap1}}$$

$$V_{\text{cell2}} = V_{\text{tap2_actual}} - V_{\text{tap1}}$$

$$V_{\text{cell3}} = V_{\text{tap3_actual}} - V_{\text{tap2_actual}}$$

$$V_{\text{pack}} = V_{\text{tap3_actual}}$$

Cell 2 and Cell 3 voltages are obtained by SUBTRACTION of adjacent tap measurements. This is called differential measurement and is the fundamental architectural principle behind how BMS ICs (like the TI BQ76920) measure cell voltages in a stack.

Resistor tolerance error analysis:

Using $\pm 1\%$ tolerance resistors for TAP2 divider (30k/10k):

$$\text{Worst-case ratio: } k_{2_max} = 10,100 / (29,700 + 10,100) = 0.2538$$

$$\text{Worst-case ratio: } k_{2_min} = 9,900 / (30,300 + 9,900) = 0.2463$$

$$\text{Ratio error: } \pm 0.75\% \rightarrow \text{At } 8.4\text{V: error} = \pm 63\text{ mV on } V_{\text{tap2}}$$

$$\text{Error on } V_{\text{cell2}}: \text{ up to } \pm(63 + 20) = \pm 83\text{ mV (sum of errors from TAP1 and TAP2)}$$

This is why using $\pm 5\%$ resistors is problematic for a BMS project:

$$\text{With } \pm 5\%: \text{ ratio error } \pm 3.75\% \rightarrow \text{at } 8.4\text{V: } \pm 315\text{ mV error on TAP2} \rightarrow \text{up to } \pm 335\text{ mV on } V_{\text{cell2}}.$$

This exceeds the 50 mV imbalance detection threshold completely.

Always use $\pm 1\%$ (or better) for voltage measurement resistors.

3.5 NTC Thermistor Design Calculations

Circuit: $5\text{V} \rightarrow R_{\text{pullup}} (10\text{k}\Omega) \rightarrow \text{junction} \rightarrow R_{\text{NTC}} (10\text{k}\Omega @ 25^\circ\text{C}) \rightarrow \text{GND}$

ADC input A3 connected at junction.

Calculating R_NTC from ADC reading:

$$V_{junction} = ADC3 \times (5.0 / 1023)$$
$$R_{NTC} = R_{pullup} \times V_{junction} / (5.0 - V_{junction})$$
$$R_{NTC} = 10,000 \times ADC3 / (1023 - ADC3)$$

Converting R_NTC to temperature (Steinhart-Hart B-parameter method):

$$1/T = 1/T_0 + (1/B) \times \ln(R_{NTC} / R_0)$$

Where:

- T = temperature in Kelvin
- T₀ = 298.15 K (25°C reference temperature)
- R₀ = 10,000 Ω (resistance at T₀)
- B = 3950 K (B-parameter, from thermistor datasheet)

Solving for T:

$$T_{kelvin} = 1 / (1/298.15 + (1/3950) \times \ln(R_{NTC} / 10000))$$
$$T_{celsius} = T_{kelvin} - 273.15$$

Verification at sample temperatures:

Temperature	Expected R_NTC	ADC3 count	Calculated T
0°C	32,650 Ω	869	0.4°C
25°C	10,000 Ω	512	25.0°C
50°C	3,602 Ω	277	49.6°C
80°C	1,669 Ω	143	79.4°C

Error is < 0.5°C across the range, acceptable for ±2°C requirement.

NTC Placement: Bond the thermistor directly to the surface of the middle cell (Cell 2) using thermal adhesive or kapton tape. This gives the best estimate of cell temperature. In production BMSs, multiple NTCs are placed at cell module level.

3.6 Hardware Failure Modes

Failure Mode	Cause	Effect on System	Detection
Resistor open circuit	PCB defect, thermal stress	ADC reads 0V → false undervoltage alarm	ADC reading stuck at 0

Failure Mode	Cause	Effect on System	Detection
Resistor short circuit	Solder bridge	ADC reads max → false overvoltage or clamp at 5V	ADC stuck at 1023
I2C bus stuck LOW	LCD glitch, missing pullup	Firmware hangs in I2C wait	Watchdog timer (add in advanced version)
NTC disconnected	Open circuit	$R_{NTC} \rightarrow \infty \rightarrow V_{\text{junction}} = 0 \rightarrow T = \text{very low}$	Temperature reads $< -40^{\circ}\text{C}$
NTC shorted	Short circuit	$R_{NTC} \rightarrow 0 \rightarrow V_{\text{junction}} = 5\text{V} \rightarrow \text{ADC3} = 1023$	ADC3 = 1023 consistently
LED open	Broken LED	Indicator fails silently	Visual inspection only
ADC noise	Insufficient decoupling	Voltage readings jitter ± 2 counts	16x oversampling reduces by 4x
Cell reversal	Incorrect wiring	$\text{TAP2} < \text{TAP1} \rightarrow V_{\text{cell2}} < 0$	Firmware checks for negative cell voltage

4. Firmware Architecture

4.1 Module Breakdown

```
main.cpp
├─ bms_adc.h / .cpp      ─ ADC acquisition with oversampling
├─ bms_voltage.h / .cpp  ─ Voltage reconstruction from tap
measurements
├─ bms_temperature.h / .cpp ─ NTC to temperature conversion
├─ bms_alarm.h / .cpp     ─ Alarm state machine
├─ bms_lcd.h / .cpp       ─ I2C LCD driver
├─ bms_uart.h / .cpp      ─ CSV UART logger
└─ bms_config.h          ─ All thresholds and constants
```

4.2 ADC Acquisition Module

16x Oversampling Implementation:

The ATmega328P ADC is 10-bit (0-1023). By taking 16 samples and summing them, the noise averages out and the effective resolution increases:

- 16 samples → sum is 0 to 16,368 → 14-bit range
- Dividing sum by 16 restores 10-bit magnitude with reduced noise (12-bit equivalent in noise terms)

Critical rule: There must be at least one ADC clock cycle between samples for the sample-and-hold capacitor to settle. The `analogRead()` function handles this internally.

```
// bms_adc.cpp
#include "bms_adc.h"

#define OVERSAMPLE_COUNT 16

uint16_t adc_read_oversampled(uint8_t pin) {
    uint32_t sum = 0;
    for (uint8_t i = 0; i < OVERSAMPLE_COUNT; i++) {
        sum += analogRead(pin);
        // Arduino analogRead() takes ~104 µs – sufficient settle time
    }
    // Divide by 16 (right shift by 4)
    return (uint16_t)(sum >> 4);
}

// Read all channels in sequence
void adc_read_all(BmsAdc_t *data) {
    data->raw_tap1 = adc_read_oversampled(PIN_TAP1);    // A0
    data->raw_tap2 = adc_read_oversampled(PIN_TAP2);    // A1
    data->raw_tap3 = adc_read_oversampled(PIN_TAP3);    // A2
    data->raw_ntc  = adc_read_oversampled(PIN_NTC);     // A3
    data->timestamp_ms = millis();
}
```

Why not just use `analogRead()` directly?

- Single reading: up to ± 2 LSB noise from ADC quantisation and board noise
- 2 LSB at 4.888 mV/count = ± 9.8 mV noise on a single reading
- 16x oversampling: noise reduces by $\sqrt{16} = 4x \rightarrow \pm 2.4$ mV equivalent
- For a BMS measuring 4.2V cells with ± 50 mV requirement, this matters

4.3 Voltage Reconstruction Module

```
// bms_voltage.cpp
#include "bms_voltage.h"

#define VREF          5.0f
#define ADC_MAX       1023.0f
#define R_TAP2_RATIO  4.0f    // (30k + 10k) / 10k
```

```

#define R_TAP3_RATIO    6.6f    // (56k + 10k) / 10k

void voltage_reconstruct(const BmsAdc_t *adc, BmsVoltage_t *volt) {
    float v_lsb = VREF / ADC_MAX;    // 4.888 mV per count

    float v_tap1 = adc->raw_tap1 * v_lsb;
    float v_tap2 = adc->raw_tap2 * v_lsb * R_TAP2_RATIO;
    float v_tap3 = adc->raw_tap3 * v_lsb * R_TAP3_RATIO;

    volt->cell1 = v_tap1;
    volt->cell2 = v_tap2 - v_tap1;
    volt->cell3 = v_tap3 - v_tap2;
    volt->pack   = v_tap3;

    // Sanity checks - catch wiring errors
    volt->valid = true;
    if (volt->cell1 < 0.0f || volt->cell2 < 0.0f || volt->cell3 < 0.0f) {
        volt->valid = false;    // Negative cell voltage = wiring error
    }
    if (volt->pack > 13.0f) {
        volt->valid = false;    // Pack above 3× maximum cell voltage
    }

    volt->imbalance = 0.0f;
    if (volt->valid) {
        float v_max = max(max(volt->cell1, volt->cell2), volt->cell3);
        float v_min = min(min(volt->cell1, volt->cell2), volt->cell3);
        volt->imbalance = v_max - v_min;
    }
}

```

4.4 Temperature Calculation Module

```

// bms_temperature.cpp
#include "bms_temperature.h"
#include <math.h>

#define NTC_B_PARAM    3950.0f    // B-parameter (K)
#define NTC_T0          298.15f    // Reference temp (25°C in Kelvin)
#define NTC_R0          10000.0f    // Resistance at T0 (10 kΩ)
#define NTC_RPULLUP     10000.0f    // Pullup resistor value (10 kΩ)
#define ADC_MAX         1023.0f

```

```
float temperature_from_adc(uint16_t adc_raw) {
    // Guard against divide-by-zero (open or shorted NTC)
    if (adc_raw == 0) return -99.0f; // NTC open circuit
    if (adc_raw >= 1023) return 99.0f; // NTC short circuit

    // Calculate R_NTC from ADC reading
    float r_ntc = NTC_RPULLUP * (float)adc_raw / (ADC_MAX - (float)adc_raw);

    // Steinhart-Hart B-parameter equation
    float inv_T = (1.0f / NTC_T0) + (1.0f / NTC_B_PARAM) * logf(r_ntc / NTC_T0);
    float T_kelvin = 1.0f / inv_T;
    float T_celsius = T_kelvin - 273.15f;

    return T_celsius;
}
```

Note on floating-point: The ATmega328P has no hardware FPU. Each `float` multiply takes ~2 µs, `logf()` takes ~200-400 µs. For temperature calculated at 1 Hz this is acceptable. For 10 Hz cell voltage monitoring, avoid floats in the hot path – use scaled integer arithmetic instead.

4.5 Alarm State Machine

The alarm system uses a state machine with hysteresis. Hysteresis prevents alarm chatter when a cell voltage fluctuates around a threshold.

States:

- `ALARM_NONE` – all values within normal range
- `ALARM_UNDERVOLTAGE` – one or more cells below threshold
- `ALARM_OVERVOLTAGE` – one or more cells above threshold
- `ALARM_OVERTEMP` – temperature above threshold
- `ALARM_FAULT` – measurement validity error or multiple simultaneous faults

Thresholds with hysteresis:

Condition	Enter Alarm (threshold)	Clear Alarm (hysteresis)
Undervoltage	< 3.00 V per cell	> 3.10 V per cell
Overvoltage	> 4.25 V per cell	< 4.15 V per cell
Overtemperature	> 50.0°C	< 47.0°C

Condition	Enter Alarm (threshold)	Clear Alarm (hysteresis)
Imbalance warning	> 100 mV delta	< 80 mV delta

```
// bms_alarm.cpp
```

```
typedef enum {
```

```
    ALARM_NONE          = 0x00,
```

```
    ALARM_UNDERVOLTAGE = 0x01,
```

```
    ALARM_OVERVOLTAGE  = 0x02,
```

```
    ALARM_OVERTEMP     = 0x04,
```

```
    ALARM_IMBALANCE    = 0x08,
```

```
    ALARM_FAULT        = 0x80
```

```
} AlarmFlags_t;
```

```
uint8_t alarm_evaluate(const BmsVoltage_t *volt, float temperature) {
    uint8_t flags = ALARM_NONE;
```

```
    if (!volt->valid) {
        return ALARM_FAULT;
    }
```

```
    // Check each cell
```

```
    float cells[3] = {volt->cell1, volt->cell2, volt->cell3};
```

```
    for (int i = 0; i < 3; i++) {
        if (cells[i] < THRESHOLD_UV_ENTER) flags |= ALARM_UNDERVOLTAGE;
        if (cells[i] > THRESHOLD_OV_ENTER) flags |= ALARM_OVERVOLTAGE;
    }
```

```
    if (temperature > THRESHOLD_OT_ENTER) flags |= ALARM_OVERTEMP;
    if (volt->imbalance > THRESHOLD_IMBAL) flags |= ALARM_IMBALANCE;
```

```
    return flags;
```

```
}
```

```
// Hysteresis implementation: alarm only clears when value returns past c
void alarm_update_with_hysteresis(BmsAlarm_t *alarm, uint8_t new_flags) {
    // Set bits can only be cleared if hysteresis condition is met
    // (implemented in alarm_clear_check() which runs separately)
    alarm->flags |= new_flags; // Set new alarm bits
    alarm->timestamp_ms = millis();
}
```

4.6 LCD Update Logic

Display format:

Line 1: "C1:3.72 C2:3.71"

Line 2: "C3:3.73 T:32.5C"

When alarm active:

Line 1: "C1:2.87 << LOW!"

Line 2: "UNDervoltage FLT"

Non-blocking LCD update:

```
// bms_lcd.cpp

#define LCD_UPDATE_INTERVAL_MS 500    // Update LCD every 500 ms

static uint32_t lcd_last_update = 0;

void lcd_update(const BmsVoltage_t *volt, float temp, uint8_t alarm_flags)
{
    uint32_t now = millis();
    if (now - lcd_last_update < LCD_UPDATE_INTERVAL_MS) return;    // Not yet time to update
    lcd_last_update = now;

    char line1[17], line2[17];

    if (alarm_flags == ALARM_NONE) {
        // Normal display
        snprintf(line1, sizeof(line1), "C1:%.2f C2:%.2f", volt->cell1, volt->cell2);
        if (volt->imbalance > 0.100f) {
            snprintf(line2, sizeof(line2), "C3:%.2f d:%.0fmV",
                    volt->cell3, volt->imbalance * 1000.0f);
        } else {
            snprintf(line2, sizeof(line2), "C3:%.2f T:%.1fC",
                    volt->cell3, temp);
        }
    } else {
        // Alarm display - show which cell triggered and alarm type
        // Find minimum voltage cell
        float v_min = min(min(volt->cell1, volt->cell2), volt->cell3);
        int cell_num = (volt->cell1 == v_min) ? 1 : (volt->cell2 == v_min) ? 2 : 3;

        if (alarm_flags & ALARM_UNDERVOLTAGE) {
            snprintf(line1, sizeof(line1), "C%d:%.2fV << LOW!", cell_num, v_min);
            if (alarm_flags & ALARM_FLT) {
                snprintf(line2, sizeof(line2), "UNDervoltage FLT");
            }
        }
    }
}
```



```

        snprintf(line2, sizeof(line2), "UNDERVOLT ALARM!");
    } else if (alarm_flags & ALARM_OVERVOLTAGE) {
        float v_max = max(max(volt->cell1, volt->cell2), volt->cell3)
        snprintf(line1, sizeof(line1), "C%d:%.2fV >> HIGH!", cell_num
        snprintf(line2, sizeof(line2), "OVERVOLT ALARM! ");
    } else if (alarm_flags & ALARM_OVERTEMP) {
        snprintf(line1, sizeof(line1), "T:%.1fC >> HOT! ", temp);
        snprintf(line2, sizeof(line2), "OVERTEMP ALARM! ");
    }
}

lcd_print_line(0, line1);
lcd_print_line(1, line2);
}

```

4.7 UART CSV Logging Architecture

CSV format (115200 baud, 8N1):

```

# GITAM-BMS-MONITOR v1.0 | Abhishek Varma | 2026
#
timestamp_ms,V_cell1,V_cell2,V_cell3,V_pack,V_imbalance,T_celsius,alarm_flags
0,3.720,3.710,3.730,11.160,0.020,28.5,0x00
1001,3.719,3.709,3.729,11.157,0.020,28.6,0x00
2003,3.720,3.711,3.730,11.161,0.019,28.6,0x00

// bms_uart.cpp

#define LOG_INTERVAL_MS 1000    // Log at 1 Hz

static bool header_printed = false;
static uint32_t log_last_ms = 0;

void uart_log_init(void) {
    Serial.begin(115200);
    while (!Serial) {} // Wait for serial port (needed for Leonardo, not
    Serial.println(F("# GITAM-BMS-MONITOR v1.0 | Abhishek Varma | 2026"))
    Serial.println(F("# ts_ms,V1,V2,V3,Vpack,Vimbal,T_C,alarm"));
}

void uart_log_sample(const BmsVoltage_t *volt, float temp, uint8_t alarm)
    uint32_t now = millis();

```

```

if (now - log_last_ms < LOG_INTERVAL_MS) return;
log_last_ms = now;

char buf[80];
// Use dtostrf() for float-to-string on AVR (printf %f not available
char v1[7], v2[7], v3[7], vp[7], vi[7], tc[7];
dtostrf(volt->cell1, 5, 3, v1);
dtostrf(volt->cell2, 5, 3, v2);
dtostrf(volt->cell3, 5, 3, v3);
dtostrf(volt->pack, 6, 3, vp);
dtostrf(volt->imbalance, 5, 3, vi);
dtostrf(temp, 5, 1, tc);

snprintf(buf, sizeof(buf), "%lu,%s,%s,%s,%s,%s,%s,0x%02X",
        now, v1, v2, v3, vp, vi, tc, alarm);
Serial.println(buf);
}

```

Why `dtostrf()` instead of `printf()` ? The AVR libc `printf` implementation in Arduino does not support `%f` by default without linking the full floating-point library (+1.5 KB flash). `dtostrf()` converts a float to a string and is already included. In STM32 firmware you use `sprintf` or `snprintf` with `%f` because the Cortex-M has hardware FPU.

4.8 Main Loop Architecture (Non-Blocking)

```

// main.cpp - Core loop structure

void loop() {
    uint32_t now = millis();

    // --- 100 ms tasks: ADC sampling ---
    if (now - last_adc_ms >= 100) {
        last_adc_ms = now;
        adc_read_all(&adc_data);
        voltage_reconstruct(&adc_data, &voltage_data);
        temperature = temperature_from_adc(adc_data.raw_ntc);
    }

    // --- 200 ms tasks: alarm evaluation ---
    if (now - last_alarm_ms >= 200) {
        last_alarm_ms = now;
        uint8_t new_flags = alarm_evaluate(&voltage_data, temperature);
    }
}

```

```

        alarm_update_with_hysteresis(&alarm_state, new_flags);
        led_update(&alarm_state, &voltage_data);
    }

    // --- 500 ms tasks: LCD update ---
    if (now - last_lcd_ms >= 500) {
        last_lcd_ms = now;
        lcd_update(&voltage_data, temperature, alarm_state.flags);
    }

    // --- 1000 ms tasks: UART logging ---
    if (now - last_log_ms >= 1000) {
        last_log_ms = now;
        uart_log_sample(&voltage_data, temperature, alarm_state.flags);
    }
}

```

Why this pattern matters: The `millis()` timing loop is the embedded equivalent of a tick-based RTOS scheduler. Each block of code runs at its own rate without blocking the others. This is the pattern used in production embedded firmware – the same concept as a FreeRTOS periodic task. Never call `delay()` in production embedded code; it blocks all other execution.

5. Implementation Plan

Phase 1: Voltage Monitoring Only (Estimated time: 2 days)

Objectives:

- Get all three cell voltages appearing on the Serial Monitor
- Verify divider calculations with a multimeter

Deliverables:

- Breadboard circuit with 3 voltage dividers wired to A0, A1, A2
- Arduino sketch reading all three taps and printing V_cell1, V_cell2, V_cell3 at 1 Hz
- First Serial Monitor screenshot with live values

Procedure:

1. Wire the resistor dividers one at a time, checking with multimeter before connecting to Arduino
2. For each tap, verify the voltage at the Arduino pin is < 5V with multimeter before powering Arduino

3. Upload minimal sketch (just `analogRead()` and `Serial.println()`) and verify counts change when battery voltage changes
4. Add divider coefficient and verify reconstructed voltage matches multimeter reading

Verification:

- Measure each tap voltage with multimeter
- Compare to reconstructed voltage from firmware
- Error must be < 50 mV for each cell
- Swap in a known resistor voltage source (e.g. 9V battery in series with a 1k pot) to simulate varying voltages

Common mistakes:

- Connecting the battery BEFORE verifying divider output is < 5V. Always measure with a multimeter first.
 - Floating A1/A2 when battery is not connected – add 100 k Ω pulldown to GND on each ADC pin to prevent floating input noise
 - Forgetting that Cell 2 voltage = TAP2_actual - TAP1_actual (not just TAP2_actual)
 - Using 5% tolerance resistors – acceptable for LED current limiting, not for voltage dividers in a BMS
-

Phase 2: LCD Integration (Estimated time: 1 day)

Objectives:

- Display live cell voltages on 16×2 I2C LCD

Deliverables:

- LCD wired (SDA=A4, SCL=A5, 5V, GND)
- Cell voltages displayed on LCD, updating at 0.5 Hz minimum

Procedure:

1. Wire LCD and run I2C scanner sketch to confirm address (0x27 or 0x3F)
2. Add LiquidCrystal_I2C library and confirm LCD displays test message
3. Integrate with voltage reading code

Verification:

- Change battery voltage (if using a bench supply) and confirm LCD reading changes
- Confirm display updates within 2 seconds

Common mistakes:

- Wrong I2C address: scan with I2C scanner sketch first
 - LCD showing garbage: check contrast adjustment potentiometer on PCF8574 module (turn it with a screwdriver)
 - Blocking loop: do NOT put `lcd.print()` inside a blocking delay
-

Phase 3: Alarm System (Estimated time: 1 day)

Objectives:

- Implement undervoltage, overvoltage, and imbalance alarm with LED indicators

Deliverables:

- All 4 LEDs functional (3 yellow per cell, 1 red alarm)
- Alarm triggers correctly when threshold is crossed
- Hysteresis prevents immediate re-trigger when voltage recovers

Procedure:

1. Wire LEDs with 220 Ω series resistors to D2, D3, D4 (yellow), D5 (red)
2. Calculate LED current: $I = (5V - 2V_f) / 220 = 3/220 = 13.6 \text{ mA}$ (within 40 mA pin limit)
3. Implement alarm state machine with hysteresis
4. Test by temporarily changing threshold to trigger with your current battery voltage

Verification:

- Lower UV threshold temporarily to 3.9V (above current battery voltage) → alarm triggers
- Raise threshold back above current voltage → alarm clears only after hysteresis band is crossed
- Verify red LED lights on any alarm state

Common mistakes:

- Setting identical enter and clear thresholds – causes rapid alarm oscillation
 - Not testing alarm with actual battery near threshold – use temporarily adjusted thresholds for testing
 - Wiring LED in reverse polarity (anode and cathode swapped) – LED simply won't light
-

Phase 4: Temperature Monitoring (Estimated time: 1 day)

Objectives:

- Wire NTC thermistor and display temperature on LCD
- Trigger overtemperature alarm

Deliverables:

- NTC wired to A3 with 10k Ω pullup to 5V
- Temperature displayed on LCD line 2
- Overtemperature alarm functional

Procedure:

1. Wire NTC: 5V $\rightarrow$ 10k Ω $\rightarrow$ A3 $\rightarrow$ NTC $\rightarrow$ GND
2. Upload temperature sketch and verify reading near room temperature
3. Touch NTC with fingertip – temperature should rise 5-10°C
4. Put NTC in cold water – temperature should drop toward 0°C

Verification:

- Compare NTC reading to a thermometer: error < 3°C
- Verify A3 = 1023 alarm (NTC open circuit) and A3 = 0 alarm (NTC short circuit) are handled in firmware

Common mistakes:

- Using wrong B-parameter value – check your thermistor datasheet. 3950K is typical but not universal.
 - Placing NTC far from battery – thermal lag gives late temperature readings
 - Not handling ADC = 0 or 1023 edge cases $\rightarrow$ division by zero or $\log(\infty)$ crash
-

Phase 5: UART CSV Logging (Estimated time: 0.5 day)

Objectives:

- Log CSV data at 1 Hz over UART
- Python tool reads and plots the data

Deliverables:

- CSV output visible on Serial Monitor
- Python BMS Analyser reads the file and generates a voltage plot

Procedure:

1. Open Serial Monitor at 115200 baud – verify CSV lines appear
2. Use PuTTY or RealTerm to log to a .csv file
3. Run Python BMS Analyser on the .csv file
4. Generate matplotlib plot of all three cell voltages vs time

Common mistakes:

- Mixing Serial.print() debugging with CSV output – clean output before logging
 - Not escaping the header comment lines (#) in the Python parser
 - Using String class in Arduino for CSV building – causes heap fragmentation on AVR; use snprintf() into a char array instead
-

Phase 6: System Validation (Estimated time: 1 day)

Objectives:

- Full integrated test of all functions
- Generate measurement accuracy table for README

Deliverables:

- Completed accuracy measurement table (10 measurements across full cell voltage range)
- 60-second demo video
- All GitHub documentation complete

Procedure: See Section 6 (Testing and Validation).

6. Testing and Validation

6.1 Accuracy Testing Protocol

Required equipment: Multimeter with ≥ 3.5 decimal places (e.g. UT61E)

Method:

1. Set up circuit with a variable DC supply (or use bench supply set to different voltages)
2. Measure reference voltage with multimeter
3. Record Arduino reconstructed voltage from Serial Monitor (average of 10 readings)
4. Calculate error = Arduino_reading - Multimeter_reading

Required accuracy table (include in README):

Reference V _{cell} (V)	Arduino Reading (V)	Error (V)	Error (%)
3.00	(measure)		
3.20			

Reference V _{cell} (V)	Arduino Reading (V)	Error (V)	Error (%)
3.40			
3.60			
3.70			
3.80			
4.00			
4.10			
4.20			

Acceptance: all errors < ±50 mV. If not, check resistor values with multimeter and recalculate divider coefficients.

6.2 Fault Injection Tests

Test	Method	Expected Result
Undervoltage Cell 1	Temporarily lower UV threshold to above current V _{cell1}	Yellow LED 1 blinks, Red LED on, LCD shows UNDERVOLT alarm
Overvoltage	Temporarily raise OV threshold below current voltage	Yellow LED blinks, Red LED on, LCD shows OVERVOLT alarm
Overtemperature	Temporarily lower OT threshold to 25°C	Red LED on, LCD shows OVERTEMP
NTC open circuit	Disconnect NTC from circuit	Firmware detects A3 = 0, logs -99°C, no crash
NTC short circuit	Short A3 to GND	Firmware detects A3 = 1023, logs 99°C, triggers OT alarm
Negative cell voltage	Swap TAP1 and TAP2 wires	Firmware detects volt->valid = false, logs ALARM_FAULT
I2C LCD disconnect	Remove SDA wire	LCD goes blank (I2C times out), rest of system continues

6.3 Long-Duration Stability Test

Run the system for 30 minutes continuously. Log all data to CSV.

Check the CSV for:

- Any missing samples (timestamp gaps > 1100ms at 1 Hz logging rate)

- Any implausible readings ($V_{\text{cell}} < 0$ or $> 4.5\text{V}$ without actual battery change)
 - Any timestamp overflow? (`millis()` overflows at 49.7 days – not an issue for 30 min test)
 - Memory: check SRAM usage (*Sketch uses X bytes of program storage space. Global variables use Y bytes of dynamic memory, leaving Z bytes for local variables.*). Z must be > 300 bytes to avoid stack overflow.
-

7. Engineering Documentation

7.1 README Structure

```
# Arduino 3S Li-ion Battery Management System

> Real-time cell voltage monitoring, temperature measurement, and fault
> detection for a 3-cell series lithium-ion battery pack.

## Hardware
- Arduino Uno (ATmega328P)
- 3× 18650 Li-ion cells in series (3S pack)
- Resistor divider network for voltage measurement
- 16×2 I2C LCD (PCF8574 + HD44780)
- 10k NTC thermistor for temperature
- 4× LED indicators (3 yellow per cell, 1 red alarm)

## Features
- Individual cell voltage measurement via differential tap method
- 16× ADC oversampling for improved accuracy ( $\pm 50$  mV typical)
- Non-blocking firmware using millis() timing (no delay())
- Alarm detection: undervoltage ( $< 3.0\text{V}$ ), overvoltage ( $> 4.25\text{V}$ ), overtemper
- Hysteresis on all thresholds to prevent alarm chatter
- 1 Hz UART CSV logging for data analysis

## Accuracy Measurement Results
[Accuracy table here]

## Schematic
[Schematic image here]

## Circuit Photos
[Photos here]

## Demo Video
```

```
[Link to 60-second demo video]

## Hardware Design Calculations
### Voltage Dividers
[Show your calculations]

### NTC Thermistor
[Show B-parameter calculation]

## Firmware Architecture
[Module diagram]

## Python BMS Analyser
...

## BOM (Bill of Materials)
[Table]

## Learning Outcomes
What I learned building this...

## Next Steps
Migration to STM32 using bare-metal C and register-level peripheral drive
```

7.2 Required Photographs

Photograph every stage. Use a stable surface, good lighting, no blur. These photos go directly in the README.

Photo	Content	Why It Matters
Overview	Complete assembled circuit, all connections visible	First thing recruiters see
Voltage dividers	Resistors clearly visible with colour band identification	Shows hardware design
LCD normal operation	LCD displaying live cell voltages	Functional proof
LCD alarm state	LCD showing alarm message with red LED lit	Alarm functionality
LED indicators	All 4 LEDs in various states	Status indication
NTC thermistor	Thermistor bonded to battery with labelled wiring	Thermal monitoring
Serial Monitor	CSV log lines visible in Arduino IDE	Data logging

Photo	Content	Why It Matters
Python plot	matplotlib graph of cell voltages vs time	Data analysis

7.3 Required Measurements and Tables

Include in README (not just in commit history):

- 1. Accuracy table (9 voltage points across 3.0V-4.2V range)
- 2. Temperature accuracy comparison (NTC vs thermometer at 5 points)
- 3. Alarm response time measurement (threshold crossing to LED on, in milliseconds)
- 4. UART throughput confirmation (bytes/second, no dropped characters at 115200 baud)

8. GitHub Portfolio Strategy

8.1 Repository Structure

```
arduino-bms-monitor/  
├── README.md                                ← The most important file. Must be  
excellent.  
├── firmware/  
│   ├── bms_monitor.ino                    ← Main Arduino sketch  
│   ├── bms_adc.h / .cpp  
│   ├── bms_voltage.h / .cpp  
│   ├── bms_temperature.h / .cpp  
│   ├── bms_alarm.h / .cpp  
│   ├── bms_lcd.h / .cpp  
│   ├── bms_uart.h / .cpp  
│   └── bms_config.h  
├── python-tools/  
│   ├── bms_analyser.py                    ← CSV parser and matplotlib plotter  
│   ├── requirements.txt                   ← numpy, matplotlib  
│   └── sample_data.csv                    ← Include sample data so tool works  
immediately  
├── hardware/  
│   ├── schematic.png                      ← Circuit diagram (draw in Fritzing or  
KiCad)  
│   ├── bom.csv                            ← Bill of materials  
│   └── calculations.pdf                   ← Voltage divider and NTC calculations  
└── docs/
```

	— photos/	← All circuit photos
	— plots/	← Generated data plots
	LICENSE	← MIT license

8.2 What Recruiters Actually Look At (in order)

1. **README.md** — 30 seconds. If it doesn't immediately explain what the project does and show a photo/demo, they move on.
2. **Circuit photo** — Is there real hardware? Many "BMS projects" on GitHub are simulations.
3. **firmware/bms_monitor.ino** — First 50 lines. Is the code structured or is it 500 lines in one function?
4. **Accuracy table** — Do they measure performance? This separates engineers from hobbyists.
5. **Commit history** — Was this built incrementally or dumped in one commit? Incremental = credible.

8.3 Demo Video Requirements

Length: 60-90 seconds maximum (2 minutes absolute maximum)

Content sequence:

1. (0-10s) Circuit overview: camera pans slowly across assembled breadboard, all components visible
2. (10-25s) Normal operation: LCD showing live cell voltages, narrate what you're reading ("Cell 1: 3.72V, Cell 2: 3.70V...")
3. (25-45s) Alarm demonstration: trigger undervoltage (temporarily lower threshold) — red LED lights, LCD changes, alarm state on LCD
4. (45-60s) Serial Monitor: show CSV data streaming in real time, then show Python plot

Technical requirements:

- Film in landscape orientation
- Arduino IDE Serial Monitor visible and readable
- Audio narration explaining what you're demonstrating
- Upload to YouTube (unlisted) and link from README

8.4 Commit Strategy

Build the project incrementally with meaningful commits:

```
git commit -m "Phase 1: ADC oversampling and tap voltage reconstruction"
git commit -m "Phase 1: Add divider coefficients, verify against
multimeter"
git commit -m "Phase 2: I2C LCD driver, display cell voltages on LCD"
```

```
git commit -m "Phase 3: Alarm state machine with undervoltage detection"
git commit -m "Phase 3: Add hysteresis to alarm state machine"
git commit -m "Phase 4: NTC thermistor B-parameter temperature
calculation"
git commit -m "Phase 5: UART CSV logger at 1 Hz, 115200 baud"
git commit -m "Phase 6: Accuracy validation table, update README"
```

One meaningful commit per phase. A recruiter seeing 10 incremental commits tells a better story than one monolithic commit.

9. Interview Preparation

9.1 Questions from an Embedded Systems Engineer

Q: "Walk me through how you measure Cell 2 voltage in your project."

A: "Cell 2 sits between TAP1 and TAP2 in the series stack. TAP1 is the voltage at the bottom of Cell 2 (also the top of Cell 1), referenced to GND. TAP2 is the voltage at the top of Cell 2, referenced to GND. I measure both through resistor dividers – 10kΩ lower leg with a 10kΩ series for TAP1 (no divider needed since it's under 4.2V), and 30kΩ upper / 10kΩ lower for TAP2. After scaling back using the divider ratio, $V_{cell2} = V_{tap2_actual} - V_{tap1_actual}$. This is differential measurement – the same principle used in production BMS ICs like the TI BQ76920."

Q: "Why 16x oversampling? What does it actually give you?"

A: "The ATmega328P ADC has 10-bit resolution, giving 4.88 mV per count. By summing 16 readings and dividing by 16, I reduce ADC noise by $\sqrt{16} = 4x$. The effective noise goes from ± 2 counts (± 9.8 mV) to approximately ± 0.5 counts (± 2.4 mV). This is called oversampling and it's a standard technique. It doesn't create new information – it averages out uncorrelated noise. If the noise is correlated (50 Hz mains hum on the battery line, for example), oversampling doesn't help and you need a hardware low-pass filter."

Q: "What happens if you use delay() in your main loop instead of millis()?"

A: "With delay(), the CPU is blocked – nothing else runs. If I call delay(1000) for the logging interval, the LCD update, alarm check, and ADC sample are all frozen for that second. In a BMS this is dangerous: a cell could cross the undervoltage threshold but the alarm won't fire until after the delay expires. The millis() pattern implements cooperative timing without blocking any task. This is conceptually identical to a FreeRTOS periodic task – each task runs at its own rate, interleaved with others."

Q: "How does your alarm hysteresis work and why does it matter?"

A: "My undervoltage alarm triggers when a cell drops below 3.0V. Without hysteresis, if the voltage fluctuates between 2.99V and 3.01V due to noise, the alarm would flicker on and off many times per second – damaging in a real system where the alarm might control a protection relay. With hysteresis,

the alarm only clears when the voltage rises above 3.10V (100 mV above the trigger point). This prevents oscillation around the threshold. The same concept appears in Schmitt triggers in hardware."

Q: "What is the source impedance constraint on the ADC and how does it affect your resistor choices?"

A: "The ATmega328P ADC has an internal sample-and-hold capacitor of approximately 14 pF. Per the datasheet, the source impedance must be below 10 k Ω for accurate conversion — higher impedance means the capacitor doesn't fully charge before conversion begins, causing error. My dividers present a source impedance equal to the parallel combination of the upper and lower resistors: for TAP2, that's 30k || 10k = 7.5 k Ω , which is under the 10 k Ω limit. This is why I can't just scale up to 300k/100k resistors to reduce current draw — the source impedance would be 75 k Ω and the ADC accuracy would degrade."

Q: "How would you verify your temperature measurement is accurate?"

A: "I compare the NTC reading to a reference thermometer (a cheap digital thermometer). I measure at 5 temperatures: cold water (5°C), ~~room temperature (25°C)~~, body temperature (37°C via holding the NTC), ~~hot water (50°C)~~, and boiling water (~100°C). The error should be under $\pm 2^\circ\text{C}$ across the range. The main sources of error are: the B-parameter value (if it's slightly different from my thermistor's actual B-parameter), resistor tolerance in the pullup resistor, and the 4.88 mV ADC resolution. For a $\pm 2^\circ\text{C}$ requirement, all three are manageable."

9.2 Questions from a BMS Engineer

Q: "What are the failure modes of your measurement approach? Would this work in a production EV BMS?"

A: "No — this approach has several problems for production use. First, the resistor dividers draw continuous current: TAP2 draws $8.4\text{V}/40\text{k}\Omega = 210\text{ }\mu\text{A}$ continuously, which is fine for a test setup but wastes energy in an automotive application. Production BMSs multiplex the cell connections using analog switches to measure sequentially, then disconnect. Second, the ADC reference voltage (5V from Arduino's VCC) varies with the supply voltage — using an external precision reference (e.g. 2.500V TL431) would be required for production. Third, the measurement accuracy ($\pm 50\text{ mV}$) is insufficient for state-of-charge estimation using OCV-SOC lookup — production BMSs need $\pm 5\text{ mV}$ or better. Fourth, there's no isolation between the battery stack and the microcontroller — in a high-voltage pack (400V), you need isolated measurement. The TI BQ76920 addresses all these issues."

Q: "What is cell balancing and why doesn't your BMS do it?"

A: "Cell balancing corrects the voltage imbalance that develops over charge/discharge cycles. Even perfectly matched cells diverge over time due to slightly different self-discharge rates and capacity fade. In passive balancing, the highest-voltage cell has a resistor switched across it, dissipating the excess energy as heat until all cells are equal. In active balancing, the excess charge is transferred to lower-voltage cells using inductive or capacitive circuits. My Arduino BMS detects imbalance ($V_{\text{max}} - V_{\text{min}} > 100\text{ mV}$) but doesn't implement balancing — it would require MOSFET switches per cell and significantly more firmware. This is Phase 4 of the project expansion."

Q: "How would you implement SOC estimation?"

A: "I'd start with Coulomb counting: integrate current over time to track charge in and out. $SOC = initial_SOC + \int (I_{charge} - I_{discharge}) dt / capacity_Ah$. This requires a current sensor (I'd add a shunt resistor and INA219 or INA226 IC). The accumulated integration drift is a known problem – small current measurement errors compound over time. To correct for drift, I'd use OCV-SOC lookup at rest: when the current has been zero for 30+ minutes, the terminal voltage approximately equals the open-circuit voltage, and I can look up the corresponding SOC from a calibrated table. The Kalman filter extends this by continuously weighting the Coulomb counter against the OCV estimate based on their respective measurement uncertainties – that's the approach I'd aim for in the STM32 version using the algorithm I'll study in the Toronto self-driving cars NPTEL-equivalent course."

Q: "What chemistry are your 18650 cells and why does it matter for your thresholds?"

A: "Standard NMC (Nickel Manganese Cobalt) or NCA (Nickel Cobalt Aluminum) 18650s – these are the most common. For NMC: nominal 3.6-3.7V, max charge 4.20V, min discharge 3.0V, cutoff 2.5V. I set my overvoltage alarm at 4.25V (not 4.20V) because of two factors: first, the typical 50 mV measurement error means I need to trigger slightly above the true threshold to avoid false alarms; second, 4.25V is still well within safe operation for most NMC cells. Different chemistries have different thresholds: LiFePO4 (lithium iron phosphate) charges to only 3.65V per cell and has a very flat discharge curve – the same voltage-based thresholds wouldn't apply."

9.3 Design Decision Questions

Q: "Why did you use a 30kΩ/10kΩ divider for TAP2 instead of something like 100kΩ/47kΩ?"

A: "Two constraints. First, source impedance: the parallel combination of the two resistors is the source impedance seen by the ADC. $30k \parallel 10k = 7.5\text{ k}\Omega$. If I used $100k \parallel 47k = 32\text{ k}\Omega$, this exceeds the 10 kΩ requirement in the ATmega328P datasheet for accurate ADC conversion. Second, the lower leg (10kΩ) sets the divider ratio. With 30k/10k, the ratio is 0.25, giving a maximum of 2.1V at A1 for an 8.4V input – well within the 5V ADC limit with margin. A 100k/47k would give 0.32 and a maximum of 2.7V, also fine, but the source impedance violation ruled it out."

Q: "Why do you have a 100 nF capacitor on each ADC input?"

A: "Two reasons. First, decoupling: the battery voltage divider node has some impedance, and capacitive coupling from nearby digital lines (PWM, I2C clock) can appear as noise on the ADC input. A 100 nF cap to GND forms a low-pass RC filter with the source resistance. For TAP2 (7.5 kΩ source), the cutoff frequency is $1/(2\pi \times 7500 \times 100e-9) = 212\text{ Hz}$. This attenuates 50 Hz mains noise (common when batteries are near AC wiring) by about 7×. Second, the capacitor helps charge the ADC sample-and-hold capacitor more quickly, improving settling."

Q: "Your accuracy is ±50 mV. The TI BQ76920 achieves ±1.5 mV. What is the gap and how would you close it?"

A: "Multiple sources contribute to my ±50 mV error. Resistor tolerance (±1%): worst-case ±83 mV on Cell 2 from accumulated divider errors. ADC quantisation: ±2.4 mV after 16x oversampling. Reference

voltage accuracy (Arduino VCC): ± 50 mV typical variation. Total worst-case $\approx \pm 135$ mV; typical is ± 50 mV. To close the gap toward BMS IC performance: use a precision 2.500V external reference (reduces reference error to $\pm 0.1\%$); use $\pm 0.1\%$ resistors (reduces divider error to ± 8 mV); use a delta-sigma ADC instead of SAR (eliminates quantisation as the dominant error). The BQ76920 uses internal 16-bit delta-sigma ADCs with a precision internal reference — that's how it achieves ± 1.5 mV."

9.4 Debugging Questions

Q: "You're reading 0.1V for Cell 3 voltage when you expect 3.7V. How do you debug it?"

A:

1. "First, measure V_{pack} directly with a multimeter — is it actually ~ 11.1 V? If not, it's a hardware issue before the divider.
2. Measure voltage at A2 pin with multimeter — is it close to 0? If yes, the upper $56\text{k}\Omega$ resistor may be open or connected wrong.
3. Check the reconstruction formula — did I calculate `volt->cell3 = V_tap3 - V_tap2` or accidentally write `volt->cell3 = V_tap3`?
4. Print raw ADC2 value to Serial. At 0.1V reconstructed (= 0.66V at the A2 pin after scaling by 1/6.6), ADC2 should be about 135 counts. If it's near 0, the issue is hardware (voltage not reaching A2). If ADC2 looks correct, the issue is in the reconstruction math.
5. If V_{tap3} is correct and V_{tap2} is also correct, but Cell 3 = 0.1V, then $V_{\text{tap3}} \approx V_{\text{tap2}}$ — meaning either Cell 3 is dead (0.1V), or the TAP3 divider has a problem."

Q: "Your LCD shows garbled characters on startup. What is the cause and fix?"

A: "The HD44780 controller has a specific initialization sequence with specific timing requirements. The LiquidCrystal_I2C library handles this, but problems arise if: the 5V supply to the LCD is not stable at power-on (add $100\mu\text{F}$ bulk cap near the LCD); the I2C address is wrong (run an I2C scanner); the contrast potentiometer is misadjusted (shows blank or all blocks); the PCF8574 pull-up resistors are missing (they're usually on the I2C backpack, but check). If it's a once-at-startup issue, add `delay(500)` before `lcd.begin()` to ensure the display controller has fully powered up."

10. Future Expansion

10.1 STM32 Migration Plan

Repository: `stm32-bms-monitor` (next project after this one)

The migration from Arduino to STM32 is not just a board swap — it's a rewrite in bare-metal C with register-level peripheral control.

Feature	Arduino Implementation	STM32 Bare-Metal Equivalent
ADC read	<code>analogRead(pin)</code>	Configure ADC1 regular channel, trigger conversion, poll EOC flag in ADC_SR register
I2C write	<code>Wire.beginTransmission(addr); Wire.write(data); Wire.endTransmission()</code>	Enable I2C1 peripheral in RCC_APB1ENR, configure I2C_CR2 with clock speed, use I2C_DR register for data, poll flags in I2C_SR1/SR2
UART print	<code>Serial.println(text)</code>	Configure USART2 in RCC_APB1ENR, set baud in USART_BRR, write byte to USART_DR, poll TXE flag
millis()	Built-in	Configure SysTick timer as 1ms interrupt, maintain <code>volatile uint32_t tick_ms</code> counter
PWM (for LED brightness)	<code>analogWrite(pin, duty)</code>	Configure TIM2/3 in PWM mode, set CCR register for duty cycle

Why this migration matters for Ather: Ather's firmware engineers configure STM32 peripherals at the register level. They don't use HAL (Hardware Abstraction Layer) because HAL adds overhead and obscures the hardware behaviour. Being able to write `GPIOA->ODR |= (1 << 5)` and explain *why* it works (RCC clock enabled, pin mode configured in MODER register, output data register bit set) is what Round 2 tests.

10.2 CAN Bus Integration

After the STM32 migration, add CAN bus output:

Target: Transmit BMS status messages over CAN at 500 kbps (automotive standard)

Hardware: STM32 Nucleo-64 (has bxCAN peripheral) + MCP2551 CAN transceiver IC

Message structure (arbitration IDs):

```
0x100: Cell voltages    [V_cell1(16), V_cell2(16), V_cell3(16),
V_pack(16)] scaled to 1mV/count
0x101: Temperature     [T_celsius(16), T_board(16), reserved(32)]
0x102: Alarm flags      [flags(8), reserved(56)]
```

Why this is relevant for Ather: Every ECU in the Ather 450X communicates over CAN. The BMS reports SOC, SOH, pack voltage, current, temperature, and fault states over CAN to the VCU (Vehicle Control Unit) and telematics controller. Implementing CAN on your 4S BMS project (Year 3) demonstrates this skill directly.

10.3 SOC Estimation Enhancement

Phase 1 (this project): Simple OCV-SOC lookup at rest (when I=0)

- Add INA219 current sensor module (~Rs 150)
- When current = 0 for 30 seconds, look up SOC from V_pack in a pre-calibrated table
- Table format: { (10.5V, 0%), (10.8V, 5%), (11.1V, 20%), (11.4V, 50%), (11.7V, 80%), (12.0V, 95%), (12.6V, 100%) }

Phase 2 (STM32 project): Coulomb counting with OCV correction

- Integrate current: SOC += (I × Δt) / capacity_Ah
- Correct drift using OCV lookup at rest periods

Phase 3 (4S BMS, Year 3): Linear Kalman filter

- State: SOC
- Measurement: V_OCV (when rest detected)
- Process model: SOC_k = SOC_{k-1} + (I_k × Δt) / capacity
- Measurement update: weight Coulomb counter against OCV estimate

10.4 Cell Balancing

Passive balancing (simpler, for Year 2 project):

- Add one NPN transistor (e.g. 2N2222) and one power resistor (10Ω/1W) per cell
- When V_cell > V_average + threshold (e.g. 20 mV), turn on the transistor to bleed current through the resistor
- Dissipates excess cell energy as heat
- Bleed current: I_bleed = (V_cell - V_average) / R_bleed = ~20 mV / 10Ω = 2 mA (very gentle balancing)

Active balancing (Year 3, 4S BMS with TI BQ76920):

- TI BQ76920 supports passive balancing via dedicated pin drivers
- Active balancing requires additional inductor-based circuits
- For the 4S BMS project, implement passive balancing using the BQ76920's built-in cell balance control bits in the BAL_CTRL register

10.5 Professional BMS Architecture Comparison

Parameter	This Arduino Project	TI BQ76920 Reference Design	Ather 450X BMS
Cell voltage accuracy	±50 mV	±1.5 mV	< 1 mV

Parameter	This Arduino Project	TI BQ76920 Reference Design	Ather 450X BMS
ADC resolution	10-bit (oversampled)	16-bit delta-sigma	16-bit
SOC method	OCV lookup (rest only)	Coulomb counting + KF	Kalman filter
Cell balancing	None	Passive (BQ76920 built-in)	Active (estimated)
Communication	UART CSV	SMBus/I2C to MCU	CAN to VCU
Cell count	3S	Up to 15S	~100S (in modules)
Protection	Software alarm only	Software + hardware overcurrent fuse	Multiple redundant
Thermal monitoring	1× NTC	Multiple NTCs per module	Distributed NTC array + model
Isolation	None	Partially isolated	Fully isolated (ISO 26262)
Operating temp	0-60°C	-40 to 85°C	-40 to 85°C

This comparison is what you present in an Ather interview when asked "what are the limitations of your BMS project and how would a production BMS address them?" It demonstrates that you understand both what you built and what you don't yet know how to build.